// THE BACKBONE, PART ONE

Settings: one place for what changes

Why settings live outside the code, what we can set, and how to get your key



// 05 SETTINGS

Three ways hard-coded values hurt

A key in the code

Pushed to GitHub, a key is public for good. A found key can be used by anyone.

A name in nine places

Change the model name once and you must remember the other eight.

Works on my machine

Your paths, keys and choices differ from a colleague's.

values change from computer to computer. Code should not.

// 05 SETTINGS

Environment variables, in plain words

.env file

A plain text file of NAME=value lines

→

Loaded at start

The program reads the file when it starts

→

Environment variables

Named values the program can ask for

→

The settings object

Checked, typed, and in one place

what the file holds, and what the code sees

```
# .env
APP_MODE=local
GOOGLE_API_KEY=your-gemini-api-key

# in Python
settings.app_mode        # "local"
```

// 05 SETTINGS

What .env.example offers

SETTING	WHAT IT DOES	NEEDED FOR
APP_MODE	local (default) or cloud: where data is kept	always
GOOGLE_API_KEY	Your Gemini key	both modes
VERTEX_LLM_MODEL_NAME	The chat model. Default: gemini-3.8-flash	optional
LLM_TEMPERATURE	0 means the most repeatable answers	optional
LOCAL_EMBEDDING_MODEL_NAME	Default: gemini-embedding-001	local mode
AGENT_MAX_MODEL_CALLS, _TOOL_CALLS, _RETRIES	Agent limits. Defaults: 8, 6 and 2	optional
GCP_PROJECT_ID, GCS_BUCKET_NAME, VECTOR_SEARCH_*	Your Google Cloud project, bucket and index	cloud mode

From .env.example and the README settings table.

// 05 SETTINGS

local (default)

Chat model
Gemini API, with your key.

Embeddings
gemini-embedding-001 through the Gemini API.

Vector database
Chroma, a folder on your disk: .meridian/chroma

Uploaded PDFs
A folder: .meridian/uploads/

You need
Only a Gemini key. No Google Cloud project.

cloud

Chat model
Gemini API, key from Secret Manager.

Embeddings
text-embedding-005 through Vertex AI.

Vector database
Vertex AI Vector Search, managed by Google.

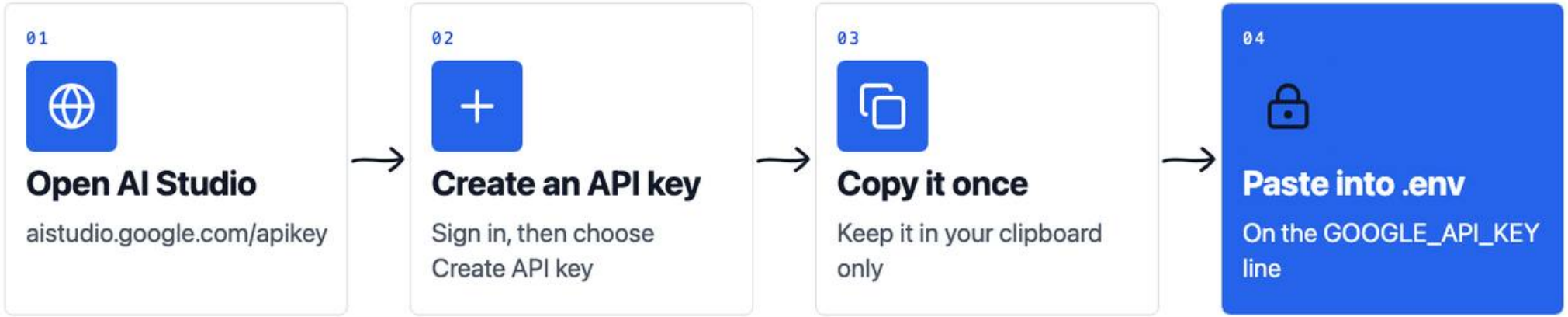
Uploaded PDFs
A Cloud Storage bucket.

You need
A Google Cloud project. Part 2 of the series.

// 05 SETTINGS

Get your Gemini API key

one free key, from Google AI Studio



Treat it like a password

Never in code, chat or screenshots

Use a throw-away key for recordings

> Free-tier content may be used by Google to improve its products, so use only the fictional sample documents.

// 05 SETTINGS

Create your .env

run from meridian-ai

```
$ cp .env.example .env
$ git status --short
# no .env in the list: Git ignores it
```

.env (the lines that matter)

```
# .env
APP_MODE=local (1)
ENVIRONMENT=local
GOOGLE_API_KEY=paste-your-key-here (2)

# the cloud block stays as it is (3)
GCP_PROJECT_ID=your-gcp-project-id
```

1. **Keep local**
   Local mode needs nothing else.

2. **Your key**
   No quotes, no spaces around the equals sign.

3. **Leave the cloud block**
   It is ignored in local mode.

// SUMMARY

Settings: the idea in five lines

1 **Values that change live outside the code.**
No key and no model name typed into a Python file.

2 **.env holds them; .env.example is the template.**
The template is in Git, the real file never is.

3 **Environment variables are named values.**
The program reads them when it starts.

4 **APP_MODE switches the whole app.**
Local needs only a key; cloud needs a project.

5 **Your key is a password.**
Create it free, paste it once, never show it.

// THE BACKBONE, PART TWO

settings.py, chunk by chunk

The file that reads .env, checks every value, and gives the whole app one object



// 06 SETTINGS.PY

What the file must do

Read the values
From the environment and from the .env file.

Give each a default and a type
Text from .env becomes a string, a number or a choice.

Refuse wrong values
APP_MODE=hybrid must fail at start-up, not later.

Offer one shared object
Every file imports the same settings.

Explain what is missing
A clear list instead of a mystery.

// 06 SETTINGS.PY

Put the file in place

run from meridian-ai

```
$ cp ../meridian-reference/backend/config/settings.py backend/config/settings.py
$ wc -l backend/config/settings.py
117 backend/config/settings.py
```

> Windows PowerShell: Copy-Item instead of cp.

Copy it

The command fills the empty file from the reference copy next to your project.

Or type it

Follow the next slides and type each chunk.

Check the size

117 lines means the whole file arrived.

// 06 SETTINGS.PY

Chunk 1: imports and the project root

config/settings.py, lines 19 to 27

```python
import os
from pathlib import Path
from typing import Literal   3

from pydantic import Field, model_validator   2
from pydantic_settings import BaseSettings, SettingsConfigDict   1

# Project root = two levels above backend/config/
PROJECT_ROOT = Path(__file__).resolve().parents[2]   4
```

1 BaseSettings

A pydantic class that reads its fields from environment variables and the .env file.

2 Field, model_validator

Field describes one setting. model_validator checks the finished object.

3 Literal

Allows only the listed values: ideal for a switch.

4 PROJECT_ROOT

This file is in backend/config/, so two parents up is the project folder. That is how .env is found.

// 06 SETTINGS.PY

Chunk 2: the class, fields and aliases

config/settings.py, lines 30 to 43

```python
class Settings(BaseSettings):
    # --- App ---
    app_name: str = "Meridian AI - Procurement Intelligence"   1
    app_env: str = Field("local", validation_alias="ENVIRONMENT")   2
    app_version: str = "1.1.0"

    # --- The toggle ---
    app_mode: Literal["local", "cloud"] = Field("local", validation_alias="APP_MODE")   3

    # --- Gemini (used in BOTH modes for the chat model) ---
    GOOGLE_API_KEY: str = Field("", validation_alias="GOOGLE_API_KEY")   4
    llm_model_name: str = Field("gemini-3.8-flash", validation_alias="VERTEX_LLM_MODEL_NAME")
    # 0 = most repeatable answers. If the model loops or repeats itself, try 1.0 (Gemini 3 default)
    llm_temperature: float = Field(0.0, validation_alias="LLM_TEMPERATURE")   5
```

1 A field with a default

A name, a type and a value used when nothing is set.

2 An alias

Called app_env in Python, but ENVIRONMENT in .env.

3 The switch

Only local or cloud is accepted. Anything else fails at start-up.

4 No secret in code

The default key is empty. The real one comes from .env.

5 Typed

The text 0.0 in .env becomes a number, because the field is a float.

// 06 SETTINGS.PY

Chunk 3: embeddings and local storage

config/settings.py, lines 45 to 54

```python
    # --- Embeddings ---
    # Cloud: Vertex AI model. Must output 768 dims to match the Vector Search index.
    embedding_model_name: str = Field("text-embedding-005", validation_alias="VERTEX_EMBEDDING_MODEL_NAME")  (1)
    # Local: Gemini API model (API key). Chroma has no fixed size, 768 keeps both modes alike.
    local_embedding_model_name: str = Field("gemini-embedding-001", validation_alias="LOCAL_EMBEDDING_MODEL_NAME")  (2)
    embedding_dimensions: int = 768  (3)

    # --- Local mode storage ---
    local_data_dir: str = Field(str(PROJECT_ROOT / ".meridian"), validation_alias="LOCAL_DATA_DIR")  (4)
    chroma_collection: str = Field("meridian_docs", validation_alias="CHROMA_COLLECTION")  (5)
```

1 Cloud embeddings

text-embedding-005 on Vertex AI.

2 Local embeddings

gemini-embedding-001 through the Gemini API.

3 768 numbers

Both produce 768 numbers per text, so the two modes behave alike.

4 Local data folder

By default .meridian/ inside the project. Chroma and uploads live there.

5 Collection name

The name of the Chroma collection.

// 06 SETTINGS.PY

Chunk 4: the cloud block and agent limits

config/settings.py, lines 56 to 68

```python
    # --- Cloud mode (GCP) ---
    GCP_PROJECT: str = Field("", validation_alias="GCP_PROJECT_ID")  (1)
    GCP_REGION: str = Field("us-central1", validation_alias="GCP_REGION")
    GCS_BUCKET_NAME: str = Field("", validation_alias="GCS_BUCKET_NAME")  (2)
    GCS_PREFIX: str = Field("uploads/", validation_alias="GCS_PREFIX")
    gcp_service_account_path: str = Field("", validation_alias="GCP_SERVICE_ACCOUNT_PATH")
    vector_search_index_id: str = Field("", validation_alias="VECTOR_SEARCH_INDEX_ID")  (3)
    vector_search_index_endpoint_id: str = Field("", validation_alias="VECTOR_SEARCH_INDEX_ENDPOINT_ID")

    # --- Agent middleware limits ---
    agent_max_model_calls: int = Field(8, validation_alias="AGENT_MAX_MODEL_CALLS")  (4)
    agent_max_tool_calls: int = Field(6, validation_alias="AGENT_MAX_TOOL_CALLS")
    agent_max_retries: int = Field(2, validation_alias="AGENT_MAX_RETRIES")  (5)
```

1 Project and region

Your Google Cloud project and where it runs.

2 Bucket and prefix

Where the cloud copy of the PDFs lives.

3 Vector index and endpoint

The identifiers of Vertex AI Vector Search.

4 Limits on model and tool calls

Defaults 8 and 6. They stop an agent that loops.

5 Retries

Default 2: how often a failed call is tried again.

// 06 SETTINGS.PY

Chunk 5: where .env is, and the validator

config/settings.py, lines 70 to 81

```python
model_config = SettingsConfigDict(  1
    env_file=str(PROJECT_ROOT / ".env"),  # found no matter which folder you start from  2
    env_file_encoding="utf-8",
    extra="ignore",  3
)

@model_validator(mode="after")  4
def _normalise(self) -> "Settings":
    # Vertex AI Vector Search expects the prefix to end with "/"
    if self.GCS_PREFIX and not self.GCS_PREFIX.endswith("/"):
        self.GCS_PREFIX += "/"
    return self
```

1 model_config

How the Settings class itself behaves.

2 env_file

Built from PROJECT_ROOT, so .env is found whatever folder you start from.

3 extra ignore

Unknown names in .env are ignored, not errors.

4 A validator

Runs after the object is built. Here it makes sure the bucket prefix ends with a slash.

// 06 SETTINGS.PY

Chunk 6: helper properties

config/settings.py, lines 83 to 94

```python
    # --- Convenience ---
    @property
    def is_cloud(self) -> bool:
        return self.app_mode == "cloud"

    @property
    def chroma_dir(self) -> str:
        return os.path.join(self.local_data_dir, "chroma")

    @property
    def local_upload_dir(self) -> str:
        return os.path.join(self.local_data_dir, "uploads")
```

1 **@property**

Turns a method into an attribute: settings.is_cloud, with no brackets.

2 **is_cloud**

The one question every file asks: are we in the cloud?

3 **chroma_dir**

The local data folder plus chroma.

4 **local_upload_dir**

Where uploaded PDFs are kept locally.

// 06 SETTINGS.PY

Chunk 7: what is missing, and one shared object

config/settings.py, lines 96 to 107

```python
    def missing_cloud_settings(self) -> list[str]:  1
        """Names of cloud settings that are still empty (used for clear error messages)."""
        required = {  2
            "GCP_PROJECT_ID": self.GCP_PROJECT,
            "GCS_BUCKET_NAME": self.GCS_BUCKET_NAME,
            "VECTOR_SEARCH_INDEX_ID": self.vector_search_index_id,
            "VECTOR_SEARCH_INDEX_ENDPOINT_ID": self.vector_search_index_endpoint_id,
        }
        return [name for name, value in required.items() if not value]  3


settings = Settings()  4
```

1 A helper

Lists the cloud settings that are still empty.

2 The four required ones

Cloud mode cannot work without them.

3 Only the empty ones

Used to write a clear message, never a mystery.

4 One shared object

Created once here, imported everywhere else.

// 06 SETTINGS.PY

Chunk 8: cloud credentials on a laptop

config/settings.py, lines 110 to 117

```python
# Cloud mode on a laptop: point Google's client libraries at the service-account key.
# (On Cloud Run nothing is set here; the service's own identity is used.)
if settings.is_cloud and settings.gcp_service_account_path:    (1)
    key_path = Path(settings.gcp_service_account_path)
    if not key_path.is_absolute():
        key_path = PROJECT_ROOT / key_path    (2)
    if key_path.exists():
        os.environ.setdefault("GOOGLE_APPLICATION_CREDENTIALS", str(key_path))    (3)
```

1 Only in cloud mode

And only when a key file path is given. This is for a laptop.

2 Relative paths

A relative path is read from the project folder.

3 setdefault

Points Google's libraries at the key file, unless it is already set.

// 06 SETTINGS.PY

Try it: four checks

check 1 and 2

```
$ cd backend
$ uv run python -c "from config.settings import settings;
print(settings.app_mode)"
local

$ APP_MODE=hybrid uv run python -c "from config.settings import settings"
pydantic_core._pydantic_core.ValidationError: 1 validation error for
Settings
APP_MODE
  Input should be 'local' or 'cloud'
```

check 3 and 4

```
$ uv run python -c "from config.settings import settings;
print(settings.missing_cloud_settings())"
['VECTOR_SEARCH_INDEX_ID', 'VECTOR_SEARCH_INDEX_ENDPOINT_ID']

$ GCS_PREFIX=uploads uv run python -c "from config.settings import
settings; print(settings.GCS_PREFIX)"
uploads/
# the validator added the slash
```

> On Windows PowerShell, set a variable first: $env:APP_MODE="hybrid"

// 06 SETTINGS.PY

Your key works

a first call to Gemini

```
$ uv run python -c "from config.settings import settings; from langchain_google_genai
import ChatGoogleGenerativeAI as G; print(G(model=settings.llm_model_name,
google_api_key=settings.GOOGLE_API_KEY).invoke('Reply with exactly: hello').text)"
hello
```

> The settings supply the model name and the key. If the key is wrong you see an API error from Google, not a hello.

a real call to Gemini

hello
The key and the model name are both correct.

An API error
Check the key in .env: no quotes and no spaces.

Model not found
Set VERTEX_LLM_MODEL_NAME to a current model name.

// SUMMARY

settings.py in five lines

1 **One class declares every setting.**
Name, type, default and, where needed, an alias.

2 **Wrong values fail at start-up.**
APP_MODE=hybrid is refused straight away.

3 **PROJECT_ROOT finds .env from anywhere.**
No dependence on the folder you run from.

4 **Helper properties answer common questions.**
is_cloud, chroma_dir, local_upload_dir.

5 **One shared object: settings.**
Every other file will import it.

// THE BACKBONE, PART THREE

Errors and logs: when things go wrong

Clear errors for users, and a diary for developers



Without clear errors

what the user sees

```
Traceback (most recent call last):
  File "api/main.py", line 52, in ask
  File "rag/llm.py", line 21, in get_llm
pydantic_core._pydantic_core.ValidationError: ...
```

A wall of text

Which line matters? Is it a bug or a missing setting?

at 2 a.m. the audit gave a strange answer. Without a log, you can only guess.

With our own errors

what the user sees

```
503: GOOGLE_API_KEY is empty. Add it to .env
```

One sentence

It says what is wrong and what to do.

and a log line that says exactly what happened, when.

Put the three files in place

run from meridian-ai

```
# run from meridian-ai. Still in backend/? Run cd .. first
$ cp ../meridian-reference/backend/exceptions/__init__.py backend/exceptions/__init__.py
$ cp ../meridian-reference/backend/logger/__init__.py backend/logger/__init__.py
$ cp ../meridian-reference/backend/logger/custom_logger.py backend/logger/custom_logger.py
$ wc -l backend/exceptions/__init__.py backend/logger/__init__.py
backend/logger/custom_logger.py
      17 backend/exceptions/__init__.py
       5 backend/logger/__init__.py
     106 backend/logger/custom_logger.py
     128 total
```

> Windows PowerShell: Copy-Item instead of cp.

The errors

exceptions/__init__.py: our own error types.

The logger

logger/__init__.py and logger/custom_logger.py: the diary.

Check the sizes

17, 5 and 106 lines, 128 in total.

Custom errors: exceptions/__init__.py

exceptions/__init__.py, lines 8 to 17 (above them: a docstring explaining the two status codes)

```python
class MeridianError(Exception):  # 1
    """Base class for every error this app raises on purpose."""


class ConfigurationError(MeridianError):  # 2
    """A required setting is missing or the selected APP_MODE cannot start."""


class IngestionError(MeridianError):  # 3
    """A document could not be loaded, chunked or indexed."""
```

1 A base class

Every error we raise on purpose inherits from it, so one except MeridianError catches them all.

2 ConfigurationError

A setting is missing, or the chosen mode cannot start. Becomes HTTP 503 later.

3 IngestionError

A document could not be read or indexed. Becomes HTTP 422 later.

// 07 ERRORS AND LOGS

From an error to an HTTP answer

OUR ERROR	WHEN IT HAPPENS	STATUS	WHAT THE CALLER READS
ConfigurationError	A setting is missing, such as no key	503	GOOGLE_API_KEY is empty. Add it to .env
IngestionError	A PDF could not be read or indexed	422	Could not read PDF 'x.pdf': ...
Anything else	An unexpected failure	500	The error text, and a full entry in the log

Built in the API section. For now: each error type has one clear answer.

Try it: the error types

run from backend/

```
# run from backend/ (new terminal? cd backend first)
$ uv run python -c "from exceptions import MeridianError, ConfigurationError;
print(issubclass(ConfigurationError, MeridianError))"
True
```

> uv run finds the project from any subfolder, but Python finds the exceptions package only from backend/.

True

ConfigurationError is a kind of MeridianError, so one except clause catches both.

ModuleNotFoundError

Python cannot find the package: you are not in backend/. Run cd backend and try again.

An empty file

Check that the copy command filled exceptions/__init__.py.

// 07 ERRORS AND LOGS

Logging in one picture

// A PLAIN MESSAGE

print

```
Tool call finished  validate_fx_hedge  105ms
```

// A STRUCTURED LOG LINE

one JSON line

```
{"tool": "validate_fx_hedge", "duration_ms": 105,
 "timestamp": "2026-10-09T22:00:22Z",
 "level": "info", "severity": "INFO",
 "event": "Tool call finished"}
```

Searchable

Find every slow tool call by its duration_ms field.

Screen and file

The same line is printed and saved.

Cloud-ready

Cloud Logging reads severity to colour and filter lines.

The logger, part 1: add_severity

logger/custom_logger.py, lines 9 to 19

```python
def add_severity(logger, method_name, event_dict):  # 1
    """Copy the log level into a field called "severity".

    Cloud Logging (which collects Cloud Run's output) recognises "severity" in a JSON
    log line and uses it to colour and filter entries. Our own "level" field is kept
    too, so nothing that reads "level" breaks.
    """
    level = event_dict.get("level")  # 2
    if level:
        event_dict["severity"] = level.upper()  # 3
    return event_dict  # 4
```

1 A processor

A function structlog calls on every log event.

2 Read the level

info, warning or error, already added earlier in the chain.

3 Copy it as severity

Upper case. Cloud Logging uses this field name to colour and filter.

4 Return the dict

So the next processor can continue.

The logger, part 2: the class and its log file

logger/custom_logger.py, lines 22 to 36

```python
class CustomLogger:
    def __init__(self, log_dir="logs"):  1
        # Ensure logs directory exists
        self.logs_dir = os.path.join(os.getcwd(), log_dir)
        os.makedirs(self.logs_dir, exist_ok=True)  2

        # Timestamped log file (for persistence)
        self._timestamp = datetime.now().strftime('%m_%d_%Y_%H_%M_%S')  3
        log_file = f"{self._timestamp}.log"
        self.log_file_path = os.path.join(self.logs_dir, log_file)

        self._gcs_flushed = False

        # Register atexit to auto-flush logs to GCS on shutdown
        atexit.register(self.flush_to_gcs)  4
```

1 A folder called logs

Created in the folder you run the program from.

2 makedirs

Creates it if it is missing. No error if it exists.

3 One file per run

Named with the date and the time.

4 atexit

At shutdown, copy the file to the cloud bucket. Cloud mode only.

The logger, part 3: two places at once

logger/custom_logger.py, lines 38 to 54

```python
def get_logger(self, name=__file__):  # 1
    logger_name = os.path.basename(name)

    # Configure logging for console + file (both JSON)
    file_handler = logging.FileHandler(self.log_file_path)  # 2
    file_handler.setLevel(logging.INFO)
    file_handler.setFormatter(logging.Formatter("%(message)s"))  # Raw JSON lines

    console_handler = logging.StreamHandler()  # 3
    console_handler.setLevel(logging.INFO)
    console_handler.setFormatter(logging.Formatter("%(message)s"))

    logging.basicConfig(  # 4
        level=logging.INFO,
        format="%(message)s",  # Structlog will handle JSON rendering
        handlers=[console_handler, file_handler]
    )
```

1 get_logger

Builds the logger and returns it with a name.

2 The file handler

Writes every line to the log file.

3 The console handler

Prints the same lines on the screen.

4 basicConfig

Level INFO, and the format is just the message: structlog already made the JSON.

The logger, part 4: the pipeline

logger/custom_logger.py, lines 56 to 70

```python
# Configure structlog for JSON structured logging
structlog.configure(
    processors=[
        structlog.processors.TimeStamper(fmt="iso", utc=True, key="timestamp"),  1
        structlog.processors.add_log_level,  2
        add_severity,  3
        structlog.processors.format_exc_info,  # log.exception(...) -> readable traceback field  4
        structlog.processors.EventRenamer(to="event"),  5
        structlog.processors.JSONRenderer()  6
    ],
    logger_factory=structlog.stdlib.LoggerFactory(),
    cache_logger_on_first_use=True,
)

return structlog.get_logger(logger_name)
```

1 Timestamp

ISO time, in UTC.

2 Level

info, warning, error.

3 Severity

Our function from part 1.

4 Exceptions

A traceback becomes a field.

5 Event name

The message is called event.

6 JSON

Printed as one JSON line.

The logger, part 5: one shared logger

logger/__init__.py (the whole file)

```python
from .custom_logger import CustomLogger

_LOGGER_INSTANCE = CustomLogger()
GLOBAL_LOGGER = _LOGGER_INSTANCE.get_logger("MeridianAI_Backend")
```

flush_to_gcs (the first lines, cloud mode)

```python
    def flush_to_gcs(self):
        """Cloud mode only: upload the current log file to GCS. Safe to call multiple
times."""
        if self._gcs_flushed:
            return

        # Local mode: the log file in ./logs is enough, never touch Google Cloud
        if not settings.is_cloud:
            return
```

1. **One shared logger**
   Created once here. Every file imports GLOBAL_LOGGER.

2. **flush_to_gcs**
   Cloud only: uploads the log file to the bucket when the server stops.

3. **Local mode**
   It returns straight away and never touches Google Cloud.

Try it: write a log line

run from backend/

```
# run from backend/ (new terminal? cd backend first)
$ uv run python -c "from logger import GLOBAL_LOGGER as log; log.info('hello', step=3)"
{"step": 3, "timestamp": "2026-10-09T23:31:15Z", "level": "info", "severity": "INFO",
"event": "hello"}
$ ls logs
10_10_2026_05_01_15.log
```

> The logs folder appears in the folder you run from, and .gitignore keeps it out of Git.

One JSON line

Printed on screen and saved in the file.

A new file each run

Named with the date and the time.

log.error and log.exception

Add an error field, or a full traceback.

// 07 ERRORS AND LOGS

Commit the backbone

run from meridian-ai

```
$ cd ..
$ git add -A
$ git commit -m "Backbone: settings, errors, logging"
[main ed22241] Backbone: settings, errors, logging
 4 files changed, 246 insertions(+)
$ git status --short
# prints nothing: .env and logs/ are ignored
```

> Your commit code (here ed22241) will differ.

Four files

settings, exceptions, logger and the logger class: 246 lines in all.

.env stays out

git status prints nothing, so your key was not saved.

A safe point

If a later step goes wrong, you can come back to this commit.

// SUMMARY

The backbone in five lines

1 **Settings live outside the code.**
One class reads them, checks them and shares them.

2 **Our own errors say what is wrong.**
ConfigurationError and IngestionError, one base class.

3 **Logs are structured JSON lines.**
Searchable fields, on screen and in a file.

4 **severity makes logs cloud-ready.**
A copy of the level in the field cloud tools expect.

5 **The backbone is done.**
Next: connecting to Gemini and turning text into numbers.